

Table of Contents

Week 7 — Method Overriding & the super Reference

When a Child Replaces or Extends a Parent's Behavior

Lesson Overview

Hour Plan

Recap of Week 6 (5 min)

Part 1 — What Is Method Overriding? (13 min)

Open with a story (3 min)

Definition

Three Real-World Override Scenarios

What Overriding Is NOT

Which Version Runs — And When

Part 2 — The Override Declaration: Saying It Clearly (10 min)

Open with a story (2 min)

The Importance of Declaring Intent

Three Simple Rules for a Valid Override

Part 3 — Extend vs Replace: Using the Parent's Behavior First (10 min)

Open with a question (2 min)

Replace vs Extend — Two Different Strategies

The "Call Parent First" Mechanism

Sealed Behavior — When Overriding Is Not Allowed

Part 4 — Overriding vs Overloading: Two Different Ideas (10 min)

Open with a question (2 min)

Side-by-Side Comparison

The Quick Test

Part 5 — Java as Illustration (8 min)

Reading Override as a Concept

Reading as Plain English

The Power This Sets Up

Test Checkpoint

Common Misconceptions

Extension Challenges

 Basic — Override Design

● Intermediate — Override or Overload?

● Advanced — When to Replace vs Extend

Lesson Summary

Homework (Mid-Term Preparation)

Week 7 — Method Overriding & the super Reference

When a Child Replaces or Extends a Parent’s Behavior

Date: Wednesday, 15 July 2026 | **Time:** 6:30 PM – 8:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Method Overriding — a child class providing its own version of a parent action
Date & Time	Wednesday, 15 July 2026 · 6:30 PM – 8:30 PM
Duration	2 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + light illustration
Prerequisites	Week 6 — Inheritance, IS-A relationship, parent-first setup
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can explain method overriding using real-world analogies, distinguish “replace” from “extend” behavior, explain why declaring intent clearly matters, distinguish overriding from overloading, and understand what “sealed” behavior means

Hour Plan

00:00 – 00:05	Recap of Week 6 + Homework check
00:05 – 00:18	Part 1 – What Is Method Overriding?
00:18 – 00:28	Part 2 – The Override Declaration: Saying It Clearly
00:28 – 00:38	Part 3 – Extend vs Replace: Using the Parent's Behavior First
00:38 – 00:48	Part 4 – Overriding vs Overloading: Two Different Ideas
00:48 – 00:56	Part 5 – Java as Illustration (light)
00:56 – 01:00	Checkpoint + Homework

 **Google Meet tip:** Keep `Week7_Summary_Examples.md` open in a second tab — it has a completely different set of real-world stories (insurance policy and mobile app updates) ready for every part of this lesson.

Recap of Week 6 (5 min)

Ask students verbally:

1. "What is the IS-A test? Give me one real example that passes and one that fails."
→ *Pass: GraduateStudent IS-A Student. Fail: Car IS-A Engine (should be HAS-A).*
2. "Why does the parent's setup always run before the child's own setup?"
→ *The child depends on the parent's foundation. Like a building's first floor before the second.*
3. "From homework — in your Person → Student → GraduateStudent design, what does GraduateStudent inherit from Person that it never had to write itself?"
→ *name, age, email, introduce(), contact() — all the way from the top.*
4. Transition question: "If Student inherits `introduce()` from Person, and that action says 'Hi, I am [name], age [age]' — what if a Student wants to ALSO say their major? Can they change that behavior?"
→ *Yes — and that is exactly what today's lesson is about.*

Part 1 — What Is Method Overriding? (13 min)

Open with a story (3 min)

"Imagine KFC has a standard recipe for fried chicken — the same recipe used in every country. Every branch in every country starts with that recipe.

But the KFC branch in Cambodia adds a local chili sauce that is not in the original recipe — the Cambodian version of 'how to serve KFC' is slightly different from the global standard, while still being recognizably KFC.

*The Cambodian branch has **overridden** the standard serving procedure with its own version."*

Definition

Method Overriding is when a child class provides its own version of an action that the parent class already defined.

The child's version **replaces** the parent's version for that specific child type.

All other child types (if any) are unaffected — they still get the parent's original version.

Three Real-World Override Scenarios

Work through each one. Ask: "What is the parent's version? What is the child's version? Why the change?"

SCENARIO 1 – RESTAURANT FRANCHISE (serving procedure)

Parent version (global KFC): `serveOrder()` → standard packaging,
standard sauces

Child version (Cambodia KFC): `serveOrder()` → standard packaging +
local chili sauce

Why override?

The Cambodian branch needs something specific that the
global standard doesn't have – but it still IS a KFC branch.

SCENARIO 2 – UNIVERSITY PEOPLE (self-introduction)

Parent version (Person): `introduce()` → "Hi, I am Sokha, age 20"

Child version (Student): `introduce()` → "Hi, I am Sokha, age 20
I study Computer Science"

Child version (Teacher): `introduce()` → "Hi, I am Syuth, age 30
I teach OOP Concepts"

Why override?

Person's introduction is too generic for a Student or Teacher.
Each needs to say something specific about their role –
but they are still Persons doing an introduction.

SCENARIO 3 – VEHICLES (fuel cost calculation)

Parent version (Vehicle): `fuelCost(km)` → returns 0 (no default)

Child version (Car): `fuelCost(km)` → $\text{km} \times \$0.12$ per km

Child version (ElectricCar): `fuelCost(km)` → $\text{km} \times \$0.03$ per km

Why override?

The parent cannot have one correct formula for all of them.
Each child knows its OWN correct calculation.

What Overriding Is NOT

NOT OVERRIDING:

Adding a completely new action the parent doesn't have

- Student adds `study()` – Person has no `study()`
- This is just adding something new, not replacing anything

Calling the parent's action from somewhere else

- `sokha.introduce()` calls Person's version if Student hasn't overridden it – no override has happened yet
-

OVERRIDING:

The child provides its OWN version of an action the parent ALREADY HAS – with the same name and same inputs.

- Student provides `introduce()` – which Person already has
 - The child's version now runs instead of the parent's
-

Which Version Runs — And When

"This is the key question: when you ask a Student to `introduce()`, which version runs — Person's or Student's?"

THE RULE:

The version that runs is determined by the ACTUAL TYPE of the object – not what label you used to hold it.

`Student sokha = new Student("Sokha", 20, "CS")`
`sokha.introduce()` → Student's version runs

`Person p = sokha` ← same object, just a different label
`p.introduce()` → STILL Student's version runs
because the actual object IS a Student

The label doesn't matter. The actual object type does.
This idea – "which version runs is decided at the moment

the program is running, not when it was written" – is the heart of Polymorphism (Week 9).

Part 2 — The Override Declaration: Saying It Clearly (10 min)

Open with a story (2 min)

"A new employee joins a company. Company policy says all employees must submit a weekly report on Mondays. But this particular employee's job contract says: 'This employee will submit a bi-weekly report on the first Monday of each month instead.'

*The contract doesn't silently hope the employee does this — it **explicitly states** the modified arrangement. This protects everyone: the employee knows what they agreed to, and the company has a written record of the exception.*

*In OOP, when a child overrides a parent's behavior, it should **declare this intent explicitly** — not just silently do something that looks like an override but might accidentally be something else."*

The Importance of Declaring Intent

*When a child class provides its own version of an inherited action, it should mark that action with `@Override`. This declaration serves as a **written contract**: "I am intentionally replacing what my parent defined — not accidentally creating something new."*

WITHOUT DECLARING INTENT:

A developer writes `Student.Introduce()` – capital I, typo.
They think they are overriding `Person.introduce()` – lowercase i.

They are NOT. They accidentally created a BRAND NEW action named `Introduce()` (capital I), and `Person's introduce()` (lowercase i) is still being called for `Student` objects.

The student's introduction never changes. But no error appears.
The developer searches for the bug for an hour.

WITH DECLARING INTENT:

The developer writes `@Override` above the method – declaring:

"I intend to replace what my parent defined."

Now the system CHECKS: "Does the parent have an action named exactly 'Introduce' with the same inputs?"

It doesn't – Person has 'introduce' (lowercase).

An error is immediately reported: "this does not actually override anything in the parent."

The bug is caught in seconds, not an hour.

Three Simple Rules for a Valid Override

RULE 1 – SAME NAME, SAME INPUTS

The child's action must have the SAME name and the SAME type of inputs as the parent's action.

Parent: introduce()

Child: introduce() ← same name, no inputs – valid override 

Parent: introduce()

Child: introduce(String greeting) ← same name, DIFFERENT input
This is NOT an override – it is something new 

RULE 2 – SAME OR COMPATIBLE OUTPUT

If the parent's action gives back a result, the child's version must give back the same type of result (or a more specific type of the same family).

RULE 3 – NOT MORE RESTRICTED

If the parent's action was open to everyone (public), the child's version must also be at least as open.
The child cannot make a parent's public action private.

Part 3 — Extend vs Replace: Using the Parent's Behavior First (10 min)

Open with a question (2 min)

"A new government regulation says: every bank branch must follow the national banking standard procedures AND each branch can add its own local welcome greeting.

The branch manager has two choices: either completely rewrite the procedure from scratch (ignoring everything the national standard said), or use the national standard as a foundation and just add the local greeting on top.

Which is smarter? Why?"

Students will agree: build on the national standard, add only the local piece.

Replace vs Extend — Two Different Strategies

STRATEGY 1 – REPLACE COMPLETELY

The child ignores the parent's version entirely.

Writes everything from scratch.

Parent: `introduce()` → "Hi, I am [name], age [age]"

Child: `introduce()` → "I'm [name], major: [major], GPA: [gpa]"
← parent's content is gone

When to use: when the parent's version is completely wrong
for the child's needs.

STRATEGY 2 – EXTEND (build on top)

The child runs the parent's version FIRST,
then adds its own specific content on top.

Parent: `introduce()` → "Hi, I am [name], age [age]"

Child: `introduce()` → calls parent version first →
"Hi, I am [name], age [age]"
then adds:
" I study [major]"

When to use: when the parent's version is still valid
and the child just needs to ADD something.

The "Call Parent First" Mechanism

"When the child wants to build on top of the parent's behavior, it has a way to say: 'Run the parent's version of this action first, then I'll add my part.'"

HOW IT WORKS IN PRACTICE:

Person's introduce() output: "Hi, I am Sokha, age 20"
Student EXTENDS this by: calling Person's version first,
then adding: " I study CS"

Final Student output:
"Hi, I am Sokha, age 20" ← from Person's version
" I study CS" ← Student's addition

Teacher EXTENDS Person's introduce() similarly:
"Hi, I am Syuth, age 30" ← from Person's version
" I teach OOP Concepts" ← Teacher's addition

Sealed Behavior — When Overriding Is Not Allowed

"Some company policies are non-negotiable. A franchise contract might say: 'The food safety checklist MUST be followed exactly as written. No branch may modify this procedure.'"

*This is the concept of **sealed behavior** — certain actions are locked and no child class is permitted to override them. The parent has decided: 'This is fixed for everyone, forever.'"*

SEALED BEHAVIOR EXAMPLES:

A safety check in a medical device class:
→ Must behave identically across all device types
→ No subclass should ever change how it works
→ Sealed – no override permitted

A standardized national exam format:
→ Every school must follow the same format
→ No individual school can override this
→ Sealed – locked by the authority above

In OOP, we seal a behavior using the keyword 'final'.
A child class that tries to override a sealed action
receives an immediate error.

Part 4 — Overriding vs Overloading: Two Different Ideas (10 min)

Open with a question (2 min)

"The word 'call' in English can mean many things depending on context:

- 'Call me' — phone call
- 'Call this meeting to order' — formally start something
- 'The bird's call' — the sound it makes

Same word, completely different meanings depending on context. But there is only ONE word 'call' — you can't change what 'call' means just because you are a different person using it."

"In OOP, we have two different situations that both involve the same action name. They are NOT the same thing."

Side-by-Side Comparison

OVERRIDING	OVERLOADING
WHAT IT IS: Child provides its own version of a parent's action	Same class provides MULTIPLE actions with the same name but different inputs
SAME NAME? Yes	Yes
SAME INPUTS? Yes – must match exactly	No – inputs must DIFFER (that's the whole point)
REQUIRES INHERITANCE? Yes – must be parent and child	No – can be within one class
DECIDED WHEN? While the program is RUNNING (Java picks which version based on the actual object type)	While the code is being READ (the compiler picks which version based on the inputs you provide)
REAL-WORLD ANALOGY: Cambodia KFC overrides the global serving procedure with its own version – same action name,	A café has: "order(drinkName)" "order(drinkName, size)"

same inputs, different behavior
per branch type

"order(drinkName, size, extras)"
Same name, different inputs

The Quick Test

"Is this overriding or overloading?"

TEST 1:

Person has: introduce()

Student has: introduce() (same name, same inputs, child of Person)

→ OVERRIDING – child replacing parent's version

TEST 2:

A class has:

showInfo()

showInfo(String format)

showInfo(String format, int limit)

→ OVERLOADING – three versions of the same name in one class
each takes different inputs

TEST 3:

Person has: greet()

Student has: greet(String language)

→ This is NEITHER a true override NOR a standard overload
(different class AND different inputs)

It creates a NEW action on Student – Person's greet() is
still inherited unchanged.

Part 5 — Java as Illustration (8 min)

Reminder: Java is just how we write these concepts down. Focus on the concept, not the syntax.

Reading Override as a Concept

```
// PARENT – defines the default version of introduce()
public class Person {
    protected String name;
    protected int age;
```

```

public Person(String name, int age) {      this.name = name;      this.age = age;
}
    public void introduce() {
        System.out.println("Hi, I am " + name + " (age " + age + ")");    }
    public String getRole() {      return "Person";    }    }
// CHILD - provides its OWN version (overrides)public class Student extends Person {
    private String major;      public Student(String name, int age, String major) {
        super(name, age);      this.major = major;    }
    // "I am declaring: I intentionally replace Person's introduce()"    @Override
    public void introduce() {
        super.introduce();          // call parent's version first (EXTEND strategy)
        System.out.println(" I study " + major);    // then add my own part    }
    // "I am declaring: I replace Person's getRole() with my own answer"    @Override
    public String getRole() {
        return "Student";          // fully replaces - does NOT call super    }
}
    // ANOTHER CHILDpublic class Teacher extends Person {
    private String subject;      public Teacher(String name, int age, String subject) {
        super(name, age);      this.subject = subject;    }    @Override
    public void introduce() {      super.introduce();          // EXTEND - same structure
        System.out.println(" I teach " + subject);    }    @Override
    public String getRole() {
        return "Teacher";          // REPLACE - different result    }    }

```

Reading as Plain English

CONCEPT TRANSLATION TABLE

Java code	OOP concept meaning
@Override	"I am declaring I replace the parent's version"
super.introduce()	"Run the parent's version of this first"
getRole()	Fully replaced - the parent's "Person" answer is gone, replaced by "Student" or "Teacher"
introduce()	Extended - parent's content kept, child adds more

The Power This Sets Up

```

// Using inheritance + overriding together:
Person[] people = {
    new Student("Sokha", 20, "CS"),
    new Teacher("Syuth", 30, "OOP"),
    new Student("Dara", 22, "Engineering"),
    new Person("Chan", 45)
};

for (Person p : people) {
    p.introduce();    // each type runs ITS OWN version
}

```

```
System.out.println("Role: " + p.getRole());    System.out.println();}
```

OUTPUT:

```
Hi, I am Sokha (age 20)      ← Person's part (via super)
  I study CS                ← Student's own addition
Role: Student

Hi, I am Syuth (age 30)
  I teach OOP
Role: Teacher

Hi, I am Dara (age 22)
  I study Engineering
Role: Student

Hi, I am Chan (age 45)      ← Person has no override → runs its own
Role: Person
```

One loop. Four different outcomes. Zero if/else checks.
This is the power that Polymorphism (Week 9) formalizes.

Test Checkpoint

- ☐ Can explain method overriding in one sentence using the franchise restaurant analogy (or different real-world example)
- ☐ Can explain why declaring intent explicitly prevents silent bugs (the typo story)
- ☐ Can distinguish "extend" (call parent first, then add) from "replace" (start fresh) strategies — and give an example of when each makes sense
- ☐ Can correctly classify a given pair of actions as overriding, overloading, or neither
- ☐ Can explain what "sealed behavior" means and give a real-world example of something that should be sealed

Common Misconceptions

Misconception	Clarification
"Overriding and overloading are the same thing"	Override = same inputs, child replaces parent. Overload = same name, different inputs, decided at read-time. Completely different ideas.
"If I write @Override, it automatically overrides"	@Override is just a declaration of intent — it tells the system to CHECK that a valid override exists. The override must actually exist for the declaration to pass.
"The child must always call the parent's version first"	No — the child can choose either strategy: fully replace the parent's behavior, or extend it by calling the parent first. The choice depends on whether the parent's behavior is still useful.
"Overriding changes the parent class"	Never. Overriding only affects what runs when the child type calls that action. The parent class itself is completely unchanged.
"If I add a new input to the method, I've overridden it"	No — adding a different input creates a new action (or an overload), not an override. An override must have the exact same inputs as the parent.

Extension Challenges

● Basic — Override Design

Design a `Vehicle → Car → ElectricCar` hierarchy in plain English (no code needed):

- `Vehicle` : has `brand`, `speed` | action: `describe()` → "Vehicle: [brand], max speed [speed]km/h"
- `Car extends Vehicle` : adds `numberOfDoors` | overrides `describe()` to EXTEND Vehicle's version, adding "Doors: [numberOfDoors]"
- `ElectricCar extends Car` : adds `batteryRangeKm` | overrides `describe()` to EXTEND Car's version, adding "Range: [batteryRangeKm]km"

Write the expected output of calling `describe()` on an `ElectricCar` object.

🟡 Intermediate — Override or Overload?

For each pair of actions below, decide whether it is **overriding**, **overloading**, or **neither**. Explain your reasoning.

1. `Person` has `greet()`. `Student` has `greet()` (same inputs, child of `Person`).
2. A single `Calculator` class has `add(int a, int b)`, `add(double a, double b)`, and `add(int a, int b, int c)`.
3. `Person` has `greet()`. `Student` has `greet(String language)` (different input).
4. `BankAccount` has `checkBalance()`. `SavingsAccount` has `checkBalance()` (same inputs, child).
5. A single `Printer` class has `print(String text)` and `print(String text, int copies)`.

🔴 Advanced — When to Replace vs Extend

Consider three different `introduce()` scenarios for three child classes that all extend `Person`:

- `Student` — wants to say name, age (from parent), AND major (new info)
- `Robot` — should say "UNIT-[id] ONLINE" with no mention of name or age
- `GuestUser` — wants to say name only, NOT age (for privacy)

For each:

- Should the child **EXTEND** the parent's behavior (call parent first, add own) or **REPLACE** it entirely?
- Justify your answer in one sentence

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. **METHOD OVERRIDING** is when a child provides its own version of an action the parent already defined.
Same name. Same inputs. The child's version runs instead.
2. **DECLARE INTENT EXPLICITLY** – always mark an override clearly.
This catches typos and mismatches before they become bugs.
3. **Two strategies:**
EXTEND – call the parent's version first, then add your own
REPLACE – start fresh, ignore the parent's version entirely
Choose based on whether the parent's behavior is still useful.
4. **SEALED BEHAVIOR** – some actions should never be overridden.
Locking them prevents any child from changing them.

5. OVERRIDING vs OVERLOADING – two completely different ideas:
Override = same inputs, child replaces parent, run-time pick
Overload = different inputs, same class, read-time pick
6. Which version runs is decided by the ACTUAL object type –
not the label used to hold it.
This is the foundation for Polymorphism (Week 9).
-

Homework (Mid-Term Preparation)

Design the following system in plain English — code is optional (bonus only):

Person → Student and Teacher hierarchy with overriding:

- Person**: attributes: `name`, `age` | actions: `introduce()` (says name and age), `getRole()` (returns "Person")
- Student extends Person**: adds `major`, `gpa` | overrides:
 - `introduce()` using EXTEND strategy — keeps parent's version, adds major and GPA
 - `getRole()` using REPLACE strategy — returns "Student"
- Teacher extends Person**: adds `subject`, `yearsExperience` | overrides:
 - `introduce()` using EXTEND strategy — keeps parent's version, adds subject
 - `getRole()` using REPLACE strategy — returns "Teacher"

For each override, state clearly: EXTEND or REPLACE strategy, and why.

Then write the **expected output** for introducing 2 students and 2 teachers from a shared `Person` list — what would each person's output look like?

Concept to carry forward: This homework is the bridge to the Mid-Term. The moment you store a Student inside a Person reference and call `introduce()` — and the Student's version runs — that is **Polymorphism**. We name and formalize it fully in Week 9.

Week 7 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com